

Table of Contents

0 – Introduction.....	8
1.a – What is Programming?	8
1.a.1 – Programming Paradigms	8
1.b – Python	8
1.b.1 – What can be done with Python?.....	8
1.b.2 – What cannot be done with Python?	9
1.b.3 – Why use Python?	9
1.b.4 – Why use another language?	9
1 – Getting Started	10
1.a – Writing/Running Code.....	10
1.a.1 – Python REPL.....	10
1.a.2 – Running Python code from the command prompt.....	11
1.a.3 – Python ‘executable’	11
1.a.4 – Notebooks	12
1.b – Sending output to the screen.....	12
1.c – Getting input from the keyboard	13
1.d – Comments	13
1.e – Variables.....	13
1.e.1 – Rules for Naming Variable.....	14
1.e.2 –Naming Conventions (for this course).....	14
1.f – Some useful built-in functions	14
1.f.1 – id()	15
1.f.2 – type().....	15
1.f.3 – dir().....	16
1.f.4 – help()	17
1.f.5 – eval().....	18
2 – Datatypes	19
2.a – None	19
2.b – Numerics	20
2.b.1 – Integers	20

2.b.2 – Floating Point Numbers	22
2.b.3 – Complex Numbers.....	22
2.b.4 – Fraction	23
2.b.5 – Decimal.....	23
2.c – Sequences:.....	23
2.c.1 – Strings	23
2.c.2 – Tuple	24
2.c.3 – Range	24
2.c.4 – List.....	24
2.d – Mappings:.....	25
2.d.1 – Dictionary	25
2.e – Set types	25
2.e.1 – Set.....	26
2.f – Binary types.....	26
2.g – Enum.....	26
2.h – Checking Data Types	26
2.i – Converting Data Types	27
2.i.1 – Implicit.....	27
2.i.2 – Explicit	27
2.j – Operators	29
2.j.1 – Assignment Operators.....	29
2.j.1 – Arithmetic Operators	30
2.j.2 – Relational (Comparison) Operators	30
2.j.3 – Logical Operators	31
2.j.4 – Identity Operators	31
2.j.5 – Membership Operators.....	32
2.j.6 – Bitwise Operators.....	32
2.j.7 – Operator precedence	33
3 – Control Structures & Sequential & Conditionals.....	35
3.a – Sequential.....	35
3.b – Conditionals	36
3.b.1 – if.....	36

3.b.2 – else	37
3.b.3 – elif.....	37
3.b.4 – and.....	37
3.b.5 – or	37
3.b.6 – not	38
3.b.7 – Ternary statements	38
3.c – Comparison Operators	39
3.c.1 – Greater than (>) operator.....	39
3.c.2 – Less than (<) operator	40
3.c.3 – Greater than or equal (>=) operator	41
3.c.4 – Less than or equal (<=) operator	41
3.c.5 – Equality (==) operator.....	42
3.c.6 – Not equals (!=) operator.....	43
3.d – Logic Operators	43
3.d.1 – Logical and operator	43
3.d.2 – Logical or operator	44
3.d.3 – Logical not operator	44
3.e – Bitwise Operators.....	45
3.e.1 – Bitwise AND (&) Operator	45
3.e.2 – Bitwise OR () Operator	45
3.e.3 – Bitwise XOR (^) Operator	45
3.e.4 – Bitwise NOT (~) Operator.....	46
3.e.5 – Left Shift (<<) Operator	46
3.e.6 – Right Shift (>>) Operator	46
Best Practice.....	46
x.b.3 – Match statement.....	47
4 – Repetition.....	49
4.a – While statements	49
4.b.1 – For loop	49
4.b.1.1 – Using an enumerable	50
4.b.1.2 – Using the range object	50
4.b.1.3 – Using the Enumerate Function	51

4.b.1.4 – Nested Loop	53
While Loop	53
For Loop	54
_ in a For Loop.....	54
break	54
continue	54
pass	55
Infinite Loops.....	55
5 – Functions	56
5.a – Creating Functions.....	56
5.a.1 – Creating Functions.....	56
5.a.2 – Calling Functions	57
5.a.3 – Functions with Inputs	57
5.a.4 – Functions with Outputs	57
5.a.5 – Variable Scope	57
5.a.6 – Keyword Arguments.....	58
5.a.7 – Default Arguments	58
5.a.8 – Annotation.....	58
5.a.9 – Recursive Functions.....	60
5.a.10 – Inner Functions.....	61
5.a.11 – Higher-order Functions	62
5.a.12 – Closures	63
5.a.13 – Lambda Closures	64
5.b – Documentation	64
5.b.1 – Code Explanation	65
5.b.2 – Disabling Code.....	65
5.b.3 – To-Do Notes	65
5.b.4 – Code Maintenance	65
5.b.5 – Code Review.....	66
5.b.6 – Debugging Information	66
5.b.7 – Best Practices for Writing Comments	66
5.c – Built-in Functions.....	67

Randomisation	69
range	70
round	70
abs	70
6 – Sequences	71
Immutables	71
Mutables	72
6.a – String	72
6.a.1 – String Creation Methods	73
6.a.1 – String Concatenation	73
6.a.2 – Escaping a String.....	74
6.a.3 – Raw strings	74
6.a.4 – Combining F-Strings and raw strings.....	75
6.b – List	75
6.b.1 – Creating Lists	75
6.b.2 – Adding Lists Together.....	75
6.b.3 – Adding an Item to a List	75
The in operator	76
List Indexing	76
List Slicing	76
6.c – Common Sequence Operations.....	76
6.b.1 – min(), max() sum().....	76
6.b.1 – Length.....	77
6.b.1 – Membership test.....	77
6.b.1 – Concatenation	77
6.b.1 – Repetition.....	77
6.b.1 – Iteration.....	77
6.b.1 – Indexing.....	77
6.b.1 – Slicing	77
8 – Sets & Mappings.....	78
Set	78
Creating sets.....	78

add(x)	79
clear()	79
copy().....	79
difference(x)	79
difference_update(iter).....	80
discard(x).....	80
intersection().....	80
intersection_update()	80
isdisjoint(x)	80
issubset()	81
issuperset().....	81
pop()	81
remove().....	81
symmetric_difference().....	82
symmetric_difference_update()	82
union().....	82
update().....	82
Dictionary	83
9 – File I/O & Errors.....	84
9.a – Working with files.....	84
9.a.1 – Opening a file	84
9.a.2 – Writing to a file.....	84
9.a.3 – Reading from a file	84
9.a.3 – A more elegant way of reading/writing a file	85
Operation System File Management	85
Checking for the existence of a file	85
Deleting a file	85
Deleting a folder.....	86
Creating a folder.....	86
9.c – Errors	86
9.c.1 – Syntax Error	86
9.c.1 – Name Error	86

9.c.2 – Type Error	87
9.c.3 – Value Error	87
9.c.4 – Index Error	87
9.c.5 – Key Error	87
9.c.6 – FileNotFoundError	87
9.c.7 – Zero Division Error	88
9.c.1 – Error handling	88
10 – Packages/Modules	89
10.a – Creating your own Module	89
10.b – Creating your own Package	89
10.c – Importing/Loading a Module	90
10.c.1 – Importing specific items from a module	90
10.c.2 – Importing Everything	90
10.c.3 – Aliasing	91
10.d – Using the Module	91
10.e – Removing a Module	92
11 – Object Oriented Programming	93
Classes and Objects	93
Defining a Python Class	93
Instantiating the Class	93
The __init__ method	93
Instance Variables	94
Instance Methods	94
Class Methods	95
Class Variables	95
Class Inheritance	95

0 – INTRODUCTION

1.a – What is Programming?

Programming is the art and science of giving a computer a precise, logical set of instructions to solve a problem or perform a task. It often involves the creative process of breaking down complex problems into small, manageable steps that a machine can understand and execute.

It uses fundamental concepts such as variables, data types and controls structures to build functional software systems such as web sites, mobile apps, games, operating systems embedded systems and AI apps.

1.a.1 – Programming Paradigms

It's a set of principles that tells you which tools to prefer and how to use them together to build something sturdy and reliable.

The main paradigms are imperative, declarative, functional and object oriented. These are covered in more details in the appendix of this document.

1.b – Python

Python is a high-level, interpreted, and general-purpose programming language. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and a simple syntax that allows programmers to express concepts in fewer lines of code than in languages like C++ or Java.

1.b.1 – What can be done with Python?

It excels in areas where developer productivity, readability, and the ability to leverage powerful libraries are more important than raw execution speed or low-level control. It is able to work seamlessly with external libraries.

- Data Science, Analytics and Visualization: This is one of Python's biggest strongholds. Libraries such as Pandas, NumPy, Matplotlib and Seaborn makes it a powerhouse for working with data.
- Machine Learning and Artificial Intelligence: Libraries like scikit-learn, TensorFlow, PyTorch and Keras are high-performant and can take advantage of specialize hardware designed for AI.
- Back-End Web Development: there are excellent libraries such as Django, Flask and FastAPI.
- Automation, Scription and DevOps
- Scientific and Numeric Computing: Access to high performance libraries like NumPy, SciPy.
- Web Scraping

- Software prototyping: It can quickly validate an idea or building a Minimum Viable Product (MVP)
- Education: It is the most popular introductory teaching language in universities, colleges and coding bootcamps worldwide

1.b.2 – What cannot be done with Python?

Although Python is a very versatile, there are areas where it is not the ideal or even a feasible tool for the job. The limitations generally fall into a few key categories: performance, system-level programming, mobile development, and certain aspects of security and memory management.

- System-level Programming such as operating systems or device drivers.
- Real-time or Ultra-High-performance Applications.
- Memory-intensive tasks in constrained environments.
- Browser/Client-Side web development.
- Statically-Typed and Secure-By-Design Systems:

1.b.3 – Why use Python?

Python is a powerful, accessible, and incredibly popular language that acts as a Swiss Army knife for programmers. Whether you're a complete beginner, a data scientist, a web developer, or a researcher, Python has the tools and community to help you get the job done efficiently. Its focus on writing clear, logical code makes it not just a great first language, but a professional tool used by companies like Google, Netflix, Instagram, and NASA.

1.b.4 – Why use another language?

- CPU intensive tasks: Python being an interpreted and high-level language is slower than compiled languages like C++.
- Mobile Apps: The eco-system is not optimized for native mobile apps. Swift, Kotlin, Java Flutter or React Native are better choices.
- Possibility of runtime errors: Python is a dynamic typed language which may be hidden errors that can be caught by compiled languages.
- In-Browser Apps: JavaScript or TypeScript

1 – GETTING STARTED

1.a – Writing/Running Code

Basically, there are four ways of running python code: from a notebook (jupyter is a local version while google colab is an online version), from the python repl, invoking the python executable and passing the code file and finally setting the permission mode of the code file. Each method has its convenience and advantage in a particular situation.

Fun fact

A Pythonista is a term used to describe an enthusiast, expert, or dedicated user of the Python programming language.

1.a.1 – Python REPL

This way is simple because it allows you to build your application incrementally while at the same time getting feedbacks in a timely manner. It is very popular with both beginners and veteran *Pythonistas*.

To use Python REPL, you will need to start the python interpreter; this exposes a command prompt similar to the one offered by most operating system shells. This is a **R**ead **E**valuate and **P**rint **L**oop. You write code blocks that are evaluated by the interpreter and the results sent back to the console.

You can use this technique to incrementally build substantial python applications or parts of applications. You are able to examine each object at your leisure. You may also debug problematic code this way.

The downside is that it is not convenient for larger/longer block of code or distributing code to end users or when typing is challenging.

Below is a sample session in the python **repl** in a windows 10 environment:

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
PS C:\progs\new100\labs> python
```

```
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1933 64  
bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> 2 + 6
```

```
8
```

```
>>> 8 / 3
```

```
2.6666666666666665
```

```
>>>  
>>> hi = 'hi '  
>>> hi = hi * 4  
>>> hi  
'hi hi hi hi '
```

1.a.2 – Running Python code from the command prompt

You will use this method only after completing your entire application and you are satisfied that your code is performing 95-100% according to specifications.

```
«path_to_python» «name_of_code_file (minus the .py extension)»
```

```
python
```

1.a.3 – Python ‘executable’

In this method of running python code there are two steps involved, changing the permission of the file and then inserting the path to the python interpreter in the first line of the code file.

```
chmod 755 «name_of_code_file»  
or  
chmod a+x «name_of_code_file»
```

And then modifying the code file, insert the following at the top of the file:

```
#!«path_name_of_python_interpreter»
```

This is probably the best and the most professional way of running or distributing your code to everyone from the novice user to the seasoned developer.

This is only possible in the Linux/Unix or MacOS operating systems that supports file permissions.

Interesting fact

Although this technique only works with *nixs OS'es, you may get the same effect in the Windows world by creating a batch or powershell script with the appropriate commands.

1.a.4 – Notebooks

This is rapidly becoming a popular way of running and demonstrating code to a wide category of users. The notebook consists of a set of three different kinds of cells: cell that contains text in markdown format, cells that contains python code and cells that shows the output from code. It is like browsing a live textbook, you can read the theory and see the code and finally view the results of executing the code without leaving the notebook. You may also modify the code and re-run to see the effects of the changes.

Online providers even have embedded environments with the required packages correctly installed and configures that can allow you to run these notebooks without any hassle. This is beneficial especially to novice user or when the environment is challenging to configure/setup more so when trying to learn new workflows such as implementing machine learning algorithm or doing image analysis.

Cloud support for notebooks (or compatible variants) includes the following entities: jupyter, Binder, Kaggle, Google, Microsoft, CoCalc and Datalore.

1.b – Sending output to the screen

The `print()` function is used to display something on the console. There are many options and as well more advanced usage that we will look at later in the semester.

```
>>> print(4)                                #displays the integer literal 4
4
>>> print(3 + 0.141592)                     #displays the float literal
3.141592
>>> print('Hello World')                   #displays the string literal
Hello World
>>> print(name)                             #display the contents of variable name
Narendra Pershad
```

1.c – Getting input from the keyboard

The `input()` function displays a prompt to the console, and then return the user response as a string. We always store the results of the `input()` function in a variable, because we will need it sometime in the future.

```
>>> name = input('What's your name? ')
What's your name? Narendra      #user response is shown in bold face
>>> print(name)
Narendra
```

1.d – Comments

Comments are lines of text that are ignored by the interpreter. They are used to improve the readability of the code for coders themselves or other developers. For single-line comments, use the “#” symbol, for multi-line comments use either triple single or double quotes.

Fun fact

Comments are not written in the repl.

```
#this is a single-line comment

'''
This is a multi-line comment.
It can span more than
one line
'''
```

Because Python is a fairly readable language, avoid frivolous comment that would distract from the logic and reduce the quality of the code. So, use comments sparingly. Some of the common uses of comments are explained below:

1.e – Variables

At the infancy of computing, when computer scientist wants to use a piece of memory, they have to use a memory address to refer to that location, this is both tedious and error-prone. Shortly afterwards, language designers allow programmers to assign a name to a memory location. So instead of referring to numeric memory addresses the programmer uses descriptive names to manipulate values.

A variable gives a name to a piece of data (or location of that data). Like a box with a label, it tells you what's inside the box.

```
name = 'Narendra'
age = 12
```

1.e.1 – Rules for Naming Variable

Variable must start with a letter or underscore. It may not contain a space nor can it be a python keyword. It may not contain a reserve symbol such as '!', '@', '\$', '#', or '%'. Avoid the use of built-in function names such as 'list', 'str', 'sum', 'int', or 'len'. Avoid starting name with double underscores.

```
>>> student_1 = 'Justin Trudeau'
>>> list = [3]                #not legal
>>> first_name = 'Narendra'    #not legal
```

1.e.2 –Naming Conventions (for this course)

Variable should be concise but descriptive, avoid single letter unless they are meaningful in the context. Use snake-casing for variable names. Use all uppercase for constants. You should keep away from starting regular variable with single or double underscores, this will avoid possible clashes with python internal names.

Fun fact

C# uses `camel_casing` for variables and `pascal_casing` for methods.

```
>>> first_name = 'Narendra'

>>> GST = 0.14                #suggest to the programmer GST is a
constant

>>> __first_name = 'Narendra'  #legal but not recommended
>>> a = 3                     #legal but not descriptive
```

1.f – Some useful built-in functions

These functions are available to use for free without you even creating them. They contain logic that is useful to have in most situations.

FYI

We will cover function formally later in this course.

1.f.1 – id()

This function returns the id of an object. (CPython uses the object's memory address). The following code runs illustrates that `int` are immutable and `list` are mutable.

```
>>> a = 3
>>> id(a)
1275421851952                                #address of the int a

>>> a = a + 7
>>> id(a)
1275421852176                                #address of the int a (different)

>>> b = []
>>> id(b)
1275429377408                                #address of the list b

>>> b.append(3)
>>> b
[3]

>>> id(b)
1275429377408                                #address of the list b (identical)
```

1.f.2 – type()

This function should be self-explanatory after examining the following code runs.

FYI

We will cover datatypes in the next chapter.

```
>>> type(3)
<class 'int'>

>>> type(3.1415)
<class 'float'>

>>> type('Narendra')
<class 'str'>
```

```
>>> type(True)
<class 'bool'>

>>> type([])
<class 'list'>

>>> type((4,))
<class 'tuple'>

>>> type({'a'})
<class 'set'>
```

1.f.3 – dir()

This function shows all the objects that are currently in scope. The first usage shows the objects in the global scope and the second usage show the objects in the `__builtins__` namespace.

FYI

A better for reflection is to use the command:
python -m pydoc -b

```
>>> dir()
['_annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'a', 'b']
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning',
'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError', 'Exception',
'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
'MemoryError', 'ModuleNotFoundError', 'NameError', 'None',
'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError',
'OverflowError', 'PendingDeprecationWarning', 'PermissionError',
'ProcessLookupError', 'RecursionError', 'ReferenceError',
```

```
'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError',
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '_',
'__build_class__', '__debug__', '__doc__', '__import__', '__loader__',
'__name__', '__package__', '__spec__', 'abs', 'aiter', 'all', 'anext',
'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes',
'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright',
'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec',
'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals',
'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance',
'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max',
'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow',
'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set',
'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super',
'tuple', 'type', 'vars', 'zip']
```

1.f.4 – help()

The launches Python built-in help system. Invoking the `help()` function with `str` and `str.split` arguments.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
 | str(object='') -> str
 | str(bytes_or_buffer[, encoding[, errors]]) -> str
 |
 | Create a new string object from the given object. If encoding or
 | errors is specified, then the object must expose a data buffer
```

```
>>> help(str.split)
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)

    Return a list of the substrings in the string, using sep as the
    separator string.
```

```
sep
```

The separator used to split the string.

1.f.5 – eval()

The `eval()` is a powerful but dangerous function that must be used with utmost caution. It evaluates strings as python code and returns the result. You should only use it on input that you trust. Never use it on un-sanitized user-input as it can be easily used to execute malicious code such a deleting files or accessing sensitive data.

It should be avoided at all cost in production code.

```
>>> eval('2 + 3')
5

>>> list_string = "[1, 2, 3, 'hello', 4.5]"
>>> my_list = eval(list_string)
>>> print(my_list
[1, 2, 3, 'hello', 4.5]
>>> print(type(my_list))
```